Task Four: Natural Language Processing Sentiment Classifier

Datheon Machine Learning & Data Science Internship | Technical Report

Author: Abdul-Azeez Mahmud Gambo **Email:** mahmudabdulazeez9@gmail.com

1. Executive Summary

This technical report presents the implementation, text processing architecture, model development, and diagnostic analysis for Task Four of the Datheon ML & DS Internship. The task involved constructing an end-to-end Natural Language Processing (NLP) pipeline to ingest product and movie review records, apply text cleaning and tokenization (lowercasing, HTML stripping, punctuation removal, stopword filtering, and lemmatization), transform cleaned text into TF-IDF feature matrices, and train a Multinomial Naive Bayes classifier for sentiment classification.

> **KEY DIAGNOSTIC FINDING & ARCHITECTURAL INSIGHT**
> The initial pipeline successfully demonstrated NLTK cleaning, TF-IDF vectorization (unigrams + bigrams), and live interactive inference via a custom predict_sentiment() function. Diagnostic testing revealed a critical data-scarcity issue: training on a 10-sample toy dataset caused extreme class prior bias toward the positive class (62.5% positive prior), leading to constant positive predictions. To achieve production readiness, the report outlines an expanded scaling solution using large-scale corpora (e.g., IMDB or Amazon Reviews).

2. Detailed Problem Statement & Technical Requirements

Automated sentiment analysis enables organizations to extract real-time customer satisfaction metrics from unstructured feedback. The technical requirements for Task Four were executed as follows:

- **1. Environment & Library Setup:** Install and download required NLTK resources (stopwords, wordnet) and verify system dependencies (regex, tqdm, joblib).
- **2. Text Preprocessing & Cleaning:** Implement a regex and NLTK pipeline (html stripping, non-alpha filtering, lowercasing, stopword removal, and WordNet lemmatization).
- **3. Vectorization Strategy:** Transform unstructured text into sparse numerical representations using TfidfVectorizer with max_features=5000 and ngram_range=(1, 2).
- **4. Multi-Class Model Training:** Train a Multinomial Naive Bayes (MultinomialNB) classifier to model class-conditional word probabilities across positive, negative, and neutral sentiments.
- **5. Live Prediction Service:** Construct a live predict_sentiment() function to accept arbitrary user text input and return probability distributions across sentiment classes.

3. NLP Preprocessing & Feature Engineering Pipeline

3.1 Cleaning & Lemmatization Pipeline

Raw text data contains noise such as markup tags, digits, and uninformative grammatical stopwords. The custom clean_and_tokenize() function executes the following sequential steps:

- **1. HTML Tag Removal:** Regex pattern r'<.*?>' strips embedded HTML tags.
- **2. Non-Alphabetical Filtering:** Regex pattern r'[^a-zA-Z]' replaces punctuation, special symbols, and digits with whitespace.
- **3. Lowercasing & Tokenization:** Standardizes casing and splits text into individual word tokens.
- **4. Stopword Removal & Lemmatization:** Filters out English stopwords via NLTK corpus and applies WordNetLemmatizer to reduce words to their base morphological dictionary form for tokens longer than 2 characters.

3.2 TF-IDF Feature Extraction

To map unstructured token sequences into numerical vector spaces, scikit-learn's TfidfVectorizer was instantiated with max_features=5000 and ngram_range=(1, 2) (capturing both single words and contiguous word pairs). Term Frequency-Inverse Document Frequency (TF-IDF) penalizes words that occur across almost all documents while highlighting terms with high sentiment specificity.

4. Model Development & Live Inference Diagnostics

4.1 Model Formulation

Multinomial Naive Bayes calculates posterior probabilities using Bayes' Theorem under the assumption of conditional word independence:

$$P(\text{Class} \mid \text{Text}) \propto P(\text{Class}) \times \prod P(\text{Term}_i \mid \text{Class})$$

4.2 Live Prediction Pipeline (`predict_sentiment`)

A modular function was constructed to process live text input, transform it through the fitted TF-IDF vectorizer, and output predicted labels alongside calibrated probability percentages across all target classes.

4.3 Diagnostic Failure Analysis (Dataset Scarcity Issue)

When testing the predict_sentiment() function on unseen evaluation phrases, the model consistently predicted POSITIVE sentiment with a fixed 62.50% confidence breakdown, regardless of input text:

Test Input Phrase	Predicted Label	Confidence	Class Probability Breakdown
"negative"	POSITIVE ⚠	62.50%	Pos: 62.50%, Neg: 25.00%, Neu: 12.50%
"negative"	POSITIVE ⚠	62.50%	Pos: 62.50%, Neg: 25.00%, Neu: 12.50%

Root Cause Analysis: The underlying dataset sentiment_reviews.csv contains only 10 total rows. With an 80/20 train-test split, the training partition contained only 8 samples, heavily dominated by positive reviews (5 out of 8 = 62.5% class prior). Out-of-vocabulary (OOV) terms in new test inputs resulted in zero TF-IDF feature matches, causing the Naive Bayes model to default entirely to the prior class distribution P(Positive) = 0.625.

5. Remediation Roadmap & Production Scaling

To resolve data-scarcity limitations and upgrade the NLP pipeline to production grade, the following enhancements are recommended:

- **1. Dataset Scaling:** Train the model on large-scale sentiment corpora such as IMDB Movie Reviews (50,000 samples) or Twitter Sentiment140 (1.6M samples) to eliminate vocabulary sparsity.
- **2. Laplace Smoothing Optimization:** Tune the Naive Bayes smoothing parameter alpha (e.g., alpha=1.0) to prevent zero-probability issues for unseen tokens.
- **3. Alternative Model Architectures:** Benchmark LogisticRegression(C=1.0) and LinearSVC alongside fine-tuned Transformer models (e.g., DistilBERT) for nuanced contextual sentiment classification.
- **4. Serialization & API Integration:** Export the TfidfVectorizer and trained MultinomialNB model into a combined joblib pipeline for deployment in REST API microservices.